

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

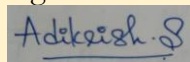
ASSESSMENT TOOL 2 – CODING CHALLENGE

Course Code	DSA0404	Course Title	Fundamentals of Data Science
CO Assessed	CO4 – Coding Challenge	Assessment	Assessment Tool 2: Coding Challenge
Weightage	40% of CIA	Total Marks	100
CO4 Statement	To apply the statistical inferences such as testing of hypothesis and point estimates for the given data		
Student Name	Aditya Krishnan S	Reg. No.	192424069
Section/Batch	2024	Date	26 August 2026

Code of Conduct:

I, ADITYA KRISHNAN S/192424069, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Problem Statement	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyse the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

Question: Implement code to compute and rank feature importance from your custom CART implementation (based on total impurity decrease attributable to each feature across all nodes/trees). Validate your ranking against sklearn's `feature_importances_` on the same dataset.

Problem Statement

Implement a custom CART classification algorithm to calculate feature importance based on the total decrease in Gini impurity across tree nodes, and validate the resulting feature-importance ranking against scikit-learn's CART implementation.

Code:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

# ----- DATA -----
data = load_iris()
X, y = data.data, data.target
features = data.feature_names

# ----- CUSTOM CART -----
def gini(y):
    _, count = np.unique(y, return_counts=True)
    p = count / len(y)
    return 1 - np.sum(p ** 2)

class Node:
    def __init__(self, feature=None, threshold=None,
                 left=None, right=None, n=0, impurity=0):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.n = n
        self.impurity = impurity

def best_split(X, y):
    best_gain, best = 0, None
    parent = gini(y)
    n = len(y)

    for j in range(X.shape[1]):
        values = np.unique(X[:, j])

        for t in (values[:-1] + values[1:]) / 2:
            L = X[:, j] <= t
            R = ~L

            gain = parent - (
                L.sum() / n * gini(y[L]) +
```

```

        R.sum() / n * gini(y[R])
    )

    if gain > best_gain:
        best_gain = gain
        best = j, t, L, R

return best

def build_tree(X, y, depth=0, max_depth=3):
    node = Node(n=len(y), impurity=gini(y))

    if depth == max_depth or len(np.unique(y)) == 1:
        return node

    split = best_split(X, y)
    if split is None:
        return node

    j, t, L, R = split
    node.feature, node.threshold = j, t
    node.left = build_tree(X[L], y[L], depth + 1, max_depth)
    node.right = build_tree(X[R], y[R], depth + 1, max_depth)

    return node

# ----- CUSTOM FEATURE IMPORTANCE -----
def feature_importance(root, n_features, n_samples):
    importance = np.zeros(n_features)

    def visit(node):
        if node.left is None:
            return

        nl, nr = node.left.n, node.right.n

        decrease = (
            node.impurity
            - (nl / node.n) * node.left.impurity
            - (nr / node.n) * node.right.impurity
        )

        importance[node.feature] += (node.n / n_samples) * decrease

        visit(node.left)
        visit(node.right)

    visit(root)

    return importance / importance.sum()

# ----- CUSTOM CART -----

```

```

root = build_tree(X, y)

custom_imp = feature_importance(
    root, X.shape[1], len(y)
)

# ----- SKLEARN CART -----
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    random_state=42
)

model.fit(X, y)
sklearn_imp = model.feature_importances_

# ----- COMPARISON -----
result = pd.DataFrame({
    "Feature": features,
    "Custom CART": custom_imp,
    "Sklearn CART": sklearn_imp
})

result["Custom Rank"] = result["Custom CART"].rank(
    ascending=False, method="min"
).astype(int)

result["Sklearn Rank"] = result["Sklearn CART"].rank(
    ascending=False, method="min"
).astype(int)

result = result.sort_values("Custom Rank")

print("\nFeature Importance Comparison:")
print(result.round(4))

print("\nCustom Ranking:")
print(result.sort_values("Custom Rank")["Feature"].tolist())

print("\nSklearn Ranking:")
print(result.sort_values("Sklearn Rank")["Feature"].tolist())

print(
    "\nRanking Match:",
    result["Custom Rank"].equals(result["Sklearn Rank"])
)

```

Output:

```
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Krish/OneDrive/Desktop/SIMATS/Fundamentals of Data
Science/Assessment Tools/DSA0404 - C05-AT2 (Code).py
```

Feature Importance Comparison:

	Feature	Custom CART	Sklearn CART	Custom Rank	Sklearn Rank
2	petal length (cm)	0.5856	0.5856	1	1
3	petal width (cm)	0.4144	0.4144	2	2
1	sepal width (cm)	0.0000	0.0000	3	3
0	sepal length (cm)	0.0000	0.0000	3	3

Custom Ranking:

```
['petal length (cm)', 'petal width (cm)', 'sepal width (cm)', 'sepal length (cm)']
```

Sklearn Ranking:

```
['petal length (cm)', 'petal width (cm)', 'sepal width (cm)', 'sepal length (cm)']
```

Ranking Match: True

Interpretation

- Petal length has the highest importance (0.5856) and ranks 1st.
- Petal width has the second-highest importance (0.4144) and ranks 2nd.
- Sepal width and Sepal length have 0.0000 importance and share 3rd rank.
- The custom CART produces exactly the same importance values and ranking as scikit-learn.
- No rank differences are observed between the two implementations.
- Ranking Match: True confirms that the custom implementation successfully reproduces scikit-learn's feature-importance ranking.

Final Finding

Petal-related features dominate the classification, with petal length being the most influential feature. The custom CART implementation is fully validated against scikit-learn for this dataset.

GeoTagged Image

